arm

# oneDNN Stack Simplification using Arm KleidiAI

RFC: #5145

Branch: jondea/replace-acl-with-kleidiai

Jonathan Deakin

June 2026

# Summary

What are we doing?

- Replacing a key component of oneDNN for AArch64

Why?

- Simpler stack ⇒ easier development
- Simpler integration into frameworks
- Better performance
- Low risk

# oneDNN + Arm Compute Library: History

## 6 years ago

Arm started optimizing oneDNN on AArch64 using Arm® Compute Library (ACL)

## 5 years ago

Fujitsu started contributing using Xbyak_aarch64 for SVE 512

- *Xbyak_aarch64 is a runtime assembler*

## 3 years ago

oneDNN + ACL became default backend for PyTorch® and TensorFlow™ on AArch64

- oneDNN was already integrated into PyTorch
- PyTorch did not want architecture specific backends

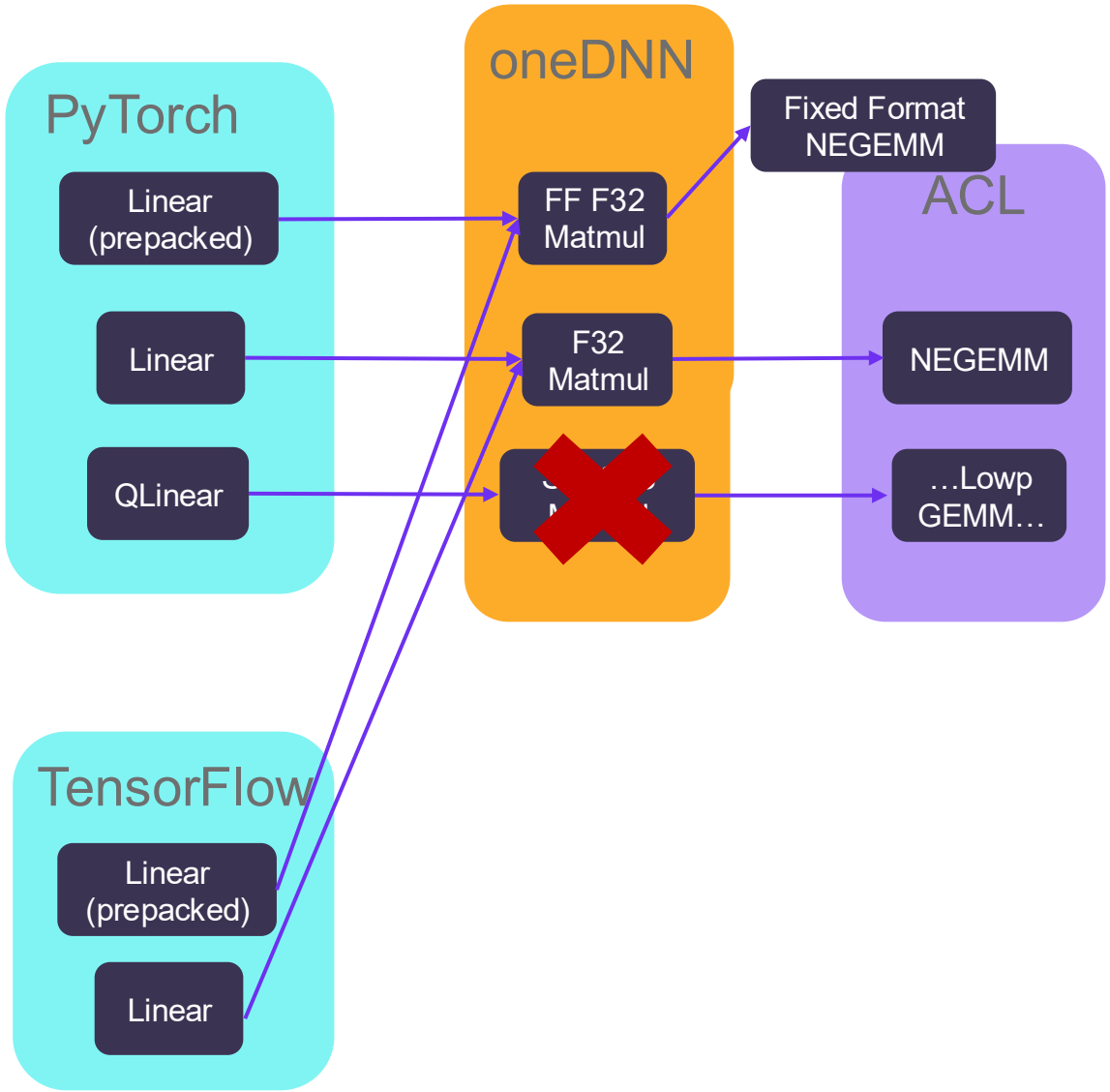
## 2 years

Fujitsu started optimizing for SVE 256

- These started to become competitive with ACL implementations

# oneDNN + ACL: History of Friction

- Initial low hanging* fruit gave **big speedups**
  (*after a hack to deal with state)
  - F32 Matmul
  - Activations
  - Quantized matmul
- Then oneDNN changed its quantization interface from configure to runtime
  - We never fixed this properly
- Kernels were added to ACL to allow frameworks to cache reorders (“Fixed Format”)
  - Took a long time to implement
  - Very complex



# oneDNN + ACL: Issues and Fix

- New features need us to design and test 2 APIs
- Extra API is an extra source of bugs
- ACL is fundamentally designed as separate framework
  - Caches reorders
  - Manages threads
  - Manages memory
  - Requires a separate build step + extra flags ⇒ awkward framework integration
- Extra state causing thread safety issues and blocking async runtime
- Despite clever workarounds, these add extra overhead
  - Threading
  - Debugging
  - **Development**

## What should we do?

- Integrate kernels more closely with oneDNN
- Make more use of Xbyak_aarch64 JIT kernels

# Replace ACL with Arm KleidiAI

# Replace ACL with KleidiAI

- Arm® KleidiAI™ is architecturally suited for integration into oneDNN
  - Specifically the library in experimental/ops
- Same as current General Matrix Multiply (GEMM) kernels inside ACL
  ⇒ low risk
  ⇒ simpler stack
- Create a lightweight GEMM object for each execution
  ⇒ no state
- GEMM object is single threaded
  ⇒ use oneDNN native thread runtime
- Can be statically linked
  ⇒ simpler build
  ⇒ simpler debug
- State of the art kernels to power oneDNN operator
  - matmul
  - convolution
  - inner product
  - deconvolution

# Benchmarks

time oneDNN + ACL / KleidiAI Ops (<1 means KleidiAI Ops is better)

| Datatype | Gops | CPU Model | Neoverse-V1 | | | | | | | | | | Neoverse-V2 | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | wtag | wtag=ab | | | | | wtag=any | | | | | wtag=ab | | | | | wtag=any | | | | |
| | | threads | 1 | 8 | 16 | 32 | 64 | 1 | 8 | 16 | 32 | 64 | 1 | 8 | 16 | 32 | 64 | 1 | 8 | 16 | 32 | 64 |
| | | desc | | | | | | | | | | | | | | | | | | | | |
| bf16:bf16:f32 | 0.004194 | 128x128:128x128 | 0.94 | 0.79 | 0.70 | 0.63 | 0.60 | 0.91 | 0.58 | 0.47 | 0.35 | 0.35 | 0.95 | 0.74 | 0.65 | 0.63 | 0.63 | 0.92 | 0.59 | 0.40 | 0.37 | 0.38 |
| | 0.033554 | 256x256:256x256 | 0.98 | 0.88 | 0.76 | 0.71 | 0.64 | 0.98 | 0.84 | 0.71 | 0.54 | 0.42 | 1.00 | 0.97 | 0.89 | 0.69 | 0.63 | 0.99 | 0.86 | 0.73 | 0.60 | 0.45 |
| | 0.268435 | 128x1024:1024x1024 | 0.99 | 0.87 | 0.93 | 0.87 | 0.73 | 0.98 | 0.97 | 0.94 | 0.87 | 0.73 | 0.99 | 0.95 | 0.99 | 0.88 | 0.75 | 0.99 | 0.97 | 0.94 | 0.87 | 0.77 |
| | | 512x512:512x512 | 0.99 | 0.99 | 0.95 | 0.95 | 0.75 | 0.98 | 0.96 | 0.94 | 0.86 | 0.70 | 1.00 | 0.99 | 0.98 | 0.92 | 0.92 | 0.99 | 0.98 | 0.94 | 0.88 | 0.77 |
| | 1.073740 | 128x1024:1024x4096 | 0.99 | 0.98 | 0.96 | 0.96 | 0.87 | 1.00 | 0.99 | 0.98 | 0.96 | 0.90 | 1.00 | 0.98 | 0.80 | 0.96 | 0.89 | 1.00 | 0.99 | 0.98 | 0.96 | 0.92 |
| | | 128x4096:4096x1024 | 0.99 | 0.99 | 0.98 | 0.96 | 0.83 | 1.00 | 0.98 | 0.98 | 0.96 | 0.90 | 1.00 | 0.99 | 0.75 | 0.96 | 0.90 | 0.99 | 0.99 | 0.97 | 0.96 | 0.92 |
| | | 512x2048:2048x512 | 0.99 | 0.96 | 0.98 | 0.97 | 0.88 | 0.99 | 0.98 | 0.98 | 0.96 | 0.90 | 0.99 | 0.98 | 0.99 | 0.97 | 0.99 | 0.99 | 0.99 | 0.98 | 0.97 | 0.91 |
| | 2.147480 | 1024x1024:1024x1024 | 0.99 | 0.97 | 0.99 | 0.99 | 0.97 | 1.00 | 0.99 | 0.99 | 0.98 | 0.95 | 0.99 | 0.99 | 1.01 | 1.00 | 0.98 | 1.00 | 0.99 | 0.99 | 0.98 | 0.96 |
| | 17.179900 | 2048x2048:2048x2048 | 0.99 | 1.00 | 0.98 | 0.96 | 0.99 | 1.00 | 0.99 | 1.00 | 1.00 | 0.99 | 0.99 | 1.00 | 0.98 | 0.97 | 0.99 | 1.00 | 0.99 | 0.99 | 0.99 | 0.99 |
| | 137.439000 | 4096x4096:4096x4096 | 1.00 | 1.00 | 0.99 | 0.99 | 0.95 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 0.96 | 0.99 | 0.99 | 1.00 | 1.01 | 1.00 |
| f32:f32:f32 | 0.004194 | 128x128:128x128 | 0.97 | 0.89 | 0.77 | 0.68 | 0.62 | 0.97 | 0.78 | 0.62 | 0.46 | 0.38 | 0.98 | 0.82 | 0.70 | 0.61 | 0.59 | 0.97 | 0.75 | 0.62 | 0.41 | 0.40 |
| | 0.033554 | 256x256:256x256 | 0.99 | 0.95 | 0.90 | 0.83 | 0.72 | 0.99 | 0.94 | 0.89 | 0.77 | 0.57 | 1.00 | 0.96 | 0.88 | 0.81 | 0.69 | 1.00 | 0.95 | 0.88 | 0.78 | 0.58 |
| | 0.268435 | 128x1024:1024x1024 | 0.99 | 0.98 | 0.98 | 0.96 | 0.87 | 1.00 | 0.98 | 0.97 | 0.95 | 0.89 | 0.99 | 0.99 | 0.99 | 0.95 | 0.89 | 1.00 | 0.99 | 0.98 | 0.96 | 0.90 |
| | | 512x512:512x512 | 0.99 | 0.99 | 0.98 | 0.97 | 0.90 | 1.00 | 0.99 | 0.98 | 0.96 | 0.89 | 1.00 | 0.99 | 0.99 | 0.97 | 0.87 | 1.00 | 0.98 | 0.99 | 0.95 | 0.89 |
| | 1.073740 | 128x1024:1024x4096 | 1.00 | 0.99 | 0.98 | 0.98 | 0.95 | 1.00 | 0.99 | 0.99 | 0.98 | 0.96 | 1.00 | 0.99 | 1.00 | 0.99 | 0.96 | 1.00 | 1.00 | 0.99 | 0.99 | 0.97 |
| | | 128x4096:4096x1024 | 0.99 | 0.99 | 0.99 | 1.00 | 0.94 | 1.00 | 0.99 | 0.99 | 0.98 | 0.97 | 0.99 | 1.00 | 1.00 | 0.99 | 0.97 | 1.00 | 1.00 | 0.99 | 0.99 | 0.97 |
| | | 512x2048:2048x512 | 1.00 | 0.99 | 0.99 | 0.98 | 0.95 | 1.00 | 0.99 | 0.99 | 0.98 | 0.97 | 1.00 | 0.97 | 0.93 | 0.99 | 0.95 | 1.00 | 0.99 | 0.99 | 0.99 | 0.97 |
| | 2.147480 | 1024x1024:1024x1024 | 1.00 | 1.00 | 1.00 | 0.99 | 0.96 | 1.00 | 0.99 | 0.99 | 0.99 | 0.98 | 1.00 | 1.00 | 0.99 | 0.99 | 0.99 | 1.00 | 1.00 | 0.99 | 0.98 | 0.98 |
| | 17.179900 | 2048x2048:2048x2048 | 1.00 | 1.01 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 0.99 | 1.00 | 1.00 | 0.99 | 0.96 | 0.93 | 1.00 | 1.00 | 1.00 | 1.00 | 0.99 |
| | 137.439000 | 4096x4096:4096x4096 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 0.99 | 1.00 | 1.00 | 1.00 | 1.00 | 0.99 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| s8:s8:f32 | 0.004194 | 128x128:128x128 | 0.74 | 0.46 | 0.40 | 0.31 | 0.30 | 0.75 | 0.46 | 0.42 | 0.30 | 0.31 | 0.76 | 0.43 | 0.38 | 0.33 | 0.32 | 0.76 | 0.43 | 0.38 | 0.32 | 0.31 |
| | 0.033554 | 256x256:256x256 | 0.93 | 0.64 | 0.55 | 0.47 | 0.35 | 0.93 | 0.63 | 0.54 | 0.48 | 0.35 | 0.92 | 0.70 | 0.61 | 0.52 | 0.42 | 0.92 | 0.70 | 0.61 | 0.53 | 0.40 |
| | 0.268435 | 128x1024:1024x1024 | 0.97 | 0.88 | 0.86 | 0.87 | 0.83 | 0.97 | 0.89 | 0.83 | 0.88 | 0.82 | 0.98 | 0.92 | 0.83 | 0.89 | 0.84 | 0.98 | 0.92 | 0.84 | 0.90 | 0.84 |
| | | 512x512:512x512 | 0.97 | 0.88 | 0.81 | 0.70 | 0.65 | 0.97 | 0.89 | 0.81 | 0.69 | 0.64 | 0.97 | 0.91 | 0.84 | 0.73 | 0.70 | 0.97 | 0.91 | 0.85 | 0.75 | 0.69 |
| | 1.073740 | 128x1024:1024x4096 | 0.99 | 0.85 | 0.77 | 0.95 | 0.92 | 0.98 | 0.84 | 0.73 | 0.95 | 0.92 | 0.98 | 0.85 | 0.79 | 0.95 | 0.94 | 0.98 | 0.83 | 0.70 | 0.96 | 0.94 |
| | | 128x4096:4096x1024 | 0.99 | 0.91 | 0.78 | 0.86 | 0.77 | 0.99 | 0.89 | 0.93 | 0.86 | 0.80 | 0.99 | 0.97 | 0.66 | 0.90 | 0.86 | 0.99 | 0.98 | 0.72 | 0.90 | 0.86 |
| | | 512x2048:2048x512 | 0.98 | 0.96 | 0.93 | 0.91 | 0.84 | 0.99 | 0.96 | 0.92 | 0.91 | 0.82 | 0.99 | 0.97 | 0.93 | 0.90 | 0.87 | 0.99 | 0.97 | 0.93 | 0.90 | 0.86 |
| | 2.147480 | 1024x1024:1024x1024 | 0.99 | 0.97 | 0.95 | 0.93 | 0.88 | 0.99 | 0.97 | 0.95 | 0.93 | 0.87 | 0.99 | 0.98 | 0.96 | 0.92 | 0.86 | 0.99 | 0.97 | 0.95 | 0.93 | 0.86 |
| | 17.179900 | 2048x2048:2048x2048 | 0.99 | 0.98 | 0.96 | 0.99 | 0.96 | 1.00 | 0.98 | 0.96 | 0.98 | 0.96 | 1.00 | 0.98 | 0.95 | 0.99 | 0.96 | 1.00 | 0.98 | 0.96 | 0.99 | 0.96 |
| | 137.439000 | 4096x4096:4096x4096 | 1.00 | 1.00 | 1.00 | 0.97 | 0.93 | 1.00 | 0.99 | 1.00 | 0.96 | 0.93 | 1.00 | 1.00 | 1.00 | 0.96 | 0.92 | 1.00 | 1.00 | 1.00 | 0.97 | 0.92 |



# All operations

| Primitive | Data Type | ASIMD Impl | SVE Impl |
|---|---|---|---|
| matmul/ip | f32/bf16/f16 | KAI ✅ | KAI ✅ |
| matmul/ip | u8/s8 | KAI ✅ | JIT ✅ [1] |
| conv | f32 | KAI ✅ [2] | JIT ✅ [2] |
| conv | f16 | KAI ✅ [2] | KAI ✅ [2] |
| conv | bf16 | KAI ✅ [2] | KAI ✅ [2] |
| conv | u8/s8 | N/A [3] | JIT ✅ |
| eltwise | all | JIT ✅ [4] | JIT ✅ |
| softmax | all | JIT ✅ [4] | JIT ✅ |
| reorder | all | JIT ✅ | JIT ✅ |
| pooling | all | JIT ✅ | JIT ✅ |
| binary | all | JIT ✅ | JIT ✅ |
| layer norm | all | JIT ✅ | JIT ✅ |
| batch norm | all | JIT ✅ | JIT ✅ |
| prelu | all | JIT ✅ | JIT ✅ |

1. KleidiAI and JIT are both good options. But JIT provides more flexibility for quant variants
2. Low risk approach: we will replace ACL convolutions like-for-like with KleidiAI
3. Not currently implemented, so no regression
4. Only log is missing (this will be fixed)

# Post-op strategy

*Post-ops allow oneDNN to fuse simple operations*

1. ACL allowed us to override the thread runtime
   a. …but the execute loop was still in ACL so post-ops could not be fused with the kernel
2. Moving to a lower level KleidiAI kernels allows us to easily fuse at the thread level
   a. Removes synchronization barrier
   b. Destination may still be in the cache
3. We may also add an endpoint to KleidiAI to fuse at the tile level to ensure destination is still in L2 cache

Ongoing work to write JIT fused post-ops, will not block release

1
```
status_t acl_matmul_t::execute() {
    _acl_kernel->execute();
    _post_ops->execute();
}
```

2
```
status_t kai_matmul_t::execute() {
    ...
    parallel([&](int ithr)) {
        _kai_kernel->execute(ithr);
        _post_ops->execute(ithr);
    });
}
```

3
```
status_t kai_matmul_t::execute() {
    ...
    parallel([&](int ithr)) {
        _kai_kernel->execute(_post_ops, ithr);
    });
}
```

# Testing and migration strategy – in progress!

- ~~Release kleidiai/experimental/ops~~ **Done!**
- Remove non-matmul ACL primitives one by one as they reach parity (~2 months). **In progress**. e.g.
  - cpu: aarch64: remove acl batch norm and layer norm implementations
  - cpu: aarch64: prelu: add jit forward
- Collect and address feedback about the PoC replace-acl-with-kleidiai branch (>2 months, we don't want to rush this). **In progress**
- Agree on an approach to integrate KleidiAI into oneDNN
- ~~Release a PyTorch PoC build for integration testing~~ **Done!**
  - docker pull armlimited/pytorch-arm-neoverse:r26.06-torch-2.13.0.dev20260616
- Merge replace-acl-with-kleidiai branch into oneDNN when approved (>2 months)
- Deal with downstream effects of removing ACL on frameworks (de-risked using Tool-Solutions) (>3 months)
  - PyTorch draft PR: https://github.com/pytorch/pytorch/pull/184372

# How to integrate KleidiAI into oneDNN?

## 1. Add KleidiAI to third party

Pros

- Simple and reliable
- Fully accelerated AArch64 build without extra build flags

Cons

- Increased source size for all platforms ~18% increase (15MB)

APPROVED

## 2. CMake FetchContent

Pros

- Updating KleidiAI is just changing a single hash
- Fully accelerated AArch64 build without extra build flags

Cons

- FetchContent not currently used in oneDNN
- FetchContent requires internet connection during configure stage

## 3. Build separately

Pros

- Similar to how we integrate ACL

Cons

- All downstream projects have extra build steps and flags
- No canonical version of KleidiAI

# Summary

What are we doing?

- Replacing ACL with KleidiAI in oneDNN for AArch64

Why?

- Simpler stack ⇒ easier development
- Simpler integration into frameworks
- Better performance
- Low risk

arm

Merci
Danke
Gracias
Grazie
谢谢
ありがとう
Asante
Thank You
감사합니다
धन्यवाद
Kiitos
شكرًا
ধন্যবাদ
תודה
ధన్యవాదములు
Köszönöm